Graph-Based Academic Library Management System: Phase II Report

Safal Shrestha, S Q Abdullah Al Mamun, Swikriti Shrestha

Abstract

This report presents the Phase II progress of the Graph-Based Academic Library Management System. The project models academic library data using a graph-oriented structure in which academic entities are represented as nodes and their relationships are represented as edges. Compared with the conceptual design from Phase I, the project has now advanced into implementation. The current system includes a detailed database schema, normalized subtype tables, multi-graph support, seeded academic data, backend query processing, and a browser-based interface for graph exploration. The prototype is implemented using SQLite, Python with Flask, and Cytoscape.js. This report expands the Phase I design by describing the detailed database design, normalization choices, schema refinements, and the current state of the implemented system.

1. Introduction

In Phase I, the project focused on understanding the problem, identifying the core entities, and designing the ER model for a graph-based academic library system. The goal was to move beyond a traditional library database and create a system capable of analyzing relationships such as author collaboration, publication citation, venue influence, and institutional affiliation.

Phase II extends that work into a functional implementation. The system now has a working schema, sample data, backend support, and an interactive frontend. At this stage, the main objective is to document how the design was translated into an actual database structure and how the current prototype supports the required graph functionality.

2. System Overview

The implemented system follows a lightweight three-layer architecture:

- Database layer: SQLite stores graph entities, relationships, and graph membership mappings.
- Backend layer: Python and Flask process requests, execute queries, and return JSON data.
- Frontend layer: HTML, CSS, JavaScript, and Cytoscape.js provide the user interface and graph visualization.

The system currently represents the following academic entities:

- Authors
- Institutions
- Publications
- Venues

Relationships between these entities are stored as edges such as:

- AUTHORED
- AFFILIATED_WITH
- PUBLISHED_IN
- CITES

- CO_AUTHOR

This design supports both structured storage and graph-based analysis.

3. Detailed Database Design

The database is centered around shared graph tables and type-specific subtype tables.

3.1 Core Graph Tables

The most important tables are Nodes and Edges.

Nodes stores the identity of every graph entity. Its main fields are:

- NodeID
- NodeType
- DisplayLabel
- CreatedAt

This table allows the system to treat all entities under a common graph abstraction.

Edges stores all relationships between nodes. Its main fields are:

- EdgeID
- SourceNodeID
- TargetNodeID
- EdgeType
- EdgeYear
- Weight
- MetadataJson

This unified edge design allows different relationship types to be handled consistently.

3.2 Subtype Tables

To avoid putting all attributes in one large generic table, the implementation uses subtype tables:

Authors

- NodeID
- FullName
- ResearchArea
- Email

Institutions

- NodeID
- Name
- Country

Publications

- NodeID
- Title
- PublicationYear
- DOI

Venues

- NodeID
- Name
- VenueKind
- Quartile
- ImpactScore

These tables store attributes specific to each entity type while keeping the graph identity in the central Nodes table.

3.3 Multi-Graph Support

To satisfy the project requirement for multi-graph membership, the system includes:

- Graphs
- NodeGraphs
- EdgeGraphs

These tables allow the same node or edge to appear in more than one graph context, such as collaboration, citation, and venue influence graphs.

4. Normalization Discussion

Normalization was important in converting the Phase I conceptual model into a stable implementation.

4.1 First Normal Form

All tables store atomic values, and each table uses a primary key. This avoids repeated groups and inconsistent row formats.

4.2 Second Normal Form

Entity-specific attributes are fully dependent on the primary key of their own table. For example:

- publication title and DOI belong only in Publications
- institution name and country belong only in Institutions
- author information belongs only in Authors

4.3 Third Normal Form

The design avoids unnecessary duplication by storing descriptive values only once in the correct table. For example:

- venue quartile is stored once in Venues
- DOI is stored once in Publications
- institution country is stored once in Institutions

This reduces update anomalies and improves consistency.

4.4 Reason for This Approach

A single all-purpose node table would have required many nullable fields and mixed unrelated attributes together. By separating graph identity from subtype details, the schema remains cleaner, more normalized, and easier to query.

5. Schema Refinements Since Phase I

Several refinements were made when moving from design to implementation.

5.1 Venue Modeling

The original design discussed journals conceptually, but the implementation refined this into a Venues node type. This made it easier to store quartile and impact information and to support journal influence analysis.

5.2 Graph Membership Mapping

Instead of attaching a single graph identifier directly to each node or edge, the system uses NodeGraphs and EdgeGraphs. This supports true many-to-many membership and better matches the requirement for reusable graph views.

5.3 Weighted Relationships

The Edges table was refined to include Weight and MetadataJson. This makes it possible to represent stronger collaboration links and store optional relationship metadata without creating many additional tables.

5.4 Practical Stack Refinement

The implementation also refined the technical stack for simplicity and portability. SQLite was chosen for lightweight deployment, Flask for a simple backend layer, and Cytoscape.js for interactive browser-based graph visualization.

6. Current System State

The current prototype is functional and includes seeded academic data representing:

- authors
- institutions
- publications
- venues
- graph relationships among them

The web interface currently displays:

- a dashboard summary of dataset counts
- a co-author network view
- an author impact view
- a Q1 journal influence network view

This demonstrates that the system has moved beyond schema design into an actual working application state.

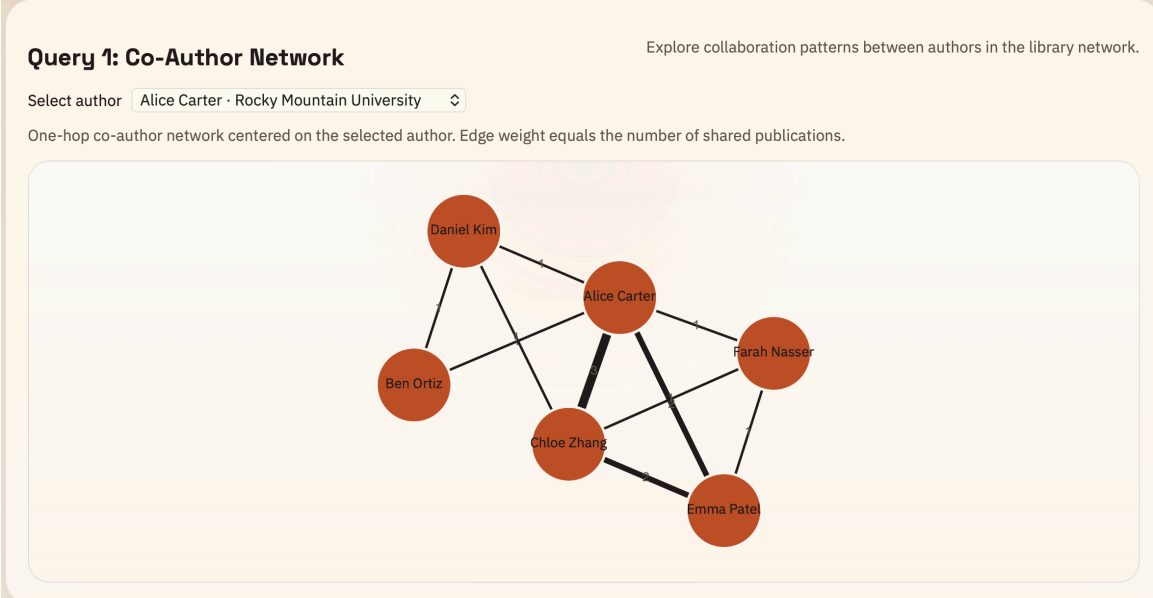

Figure 1. Current system state showing dashboard summary and dataset overview.

7. Interfaces

The prototype includes a browser-based interface designed to present the graph data clearly.

7.1 Co-Author Interface

Users can select an author from a dropdown and view the one-hop co-author network for that author. The result is displayed as an interactive graph.

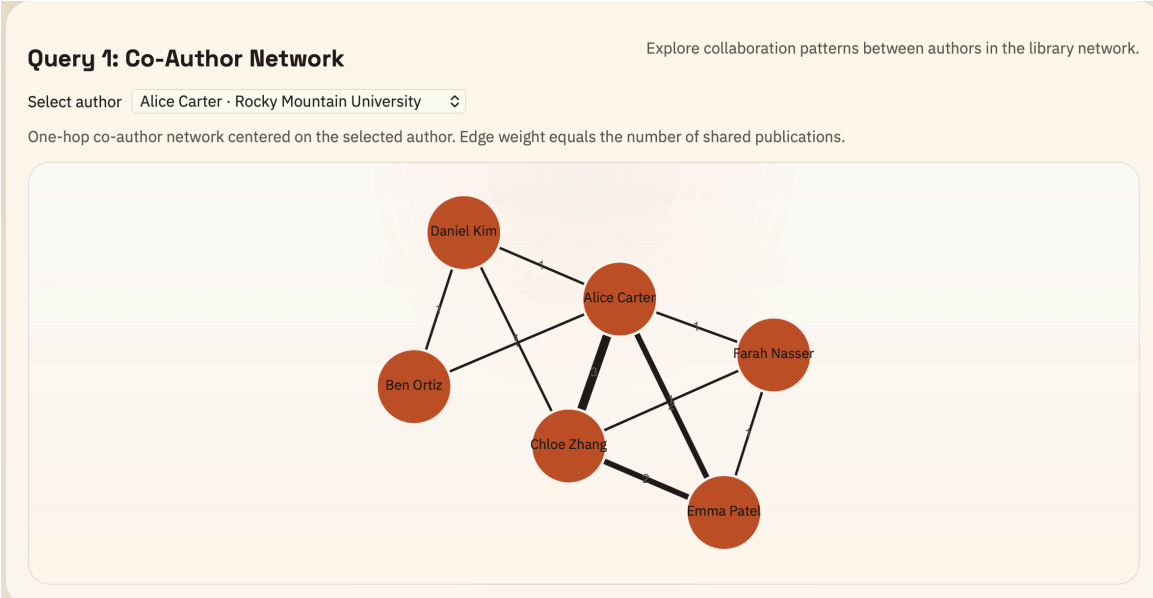

Figure 2. Co-author network interface centered on a selected author.

7.2 Author Impact Interface

The system provides a table showing authors with strong citation-based impact. The interface includes:

- author name
- impact score
- publication venues
- top publications

Internally, this section is based on the required h-index logic.

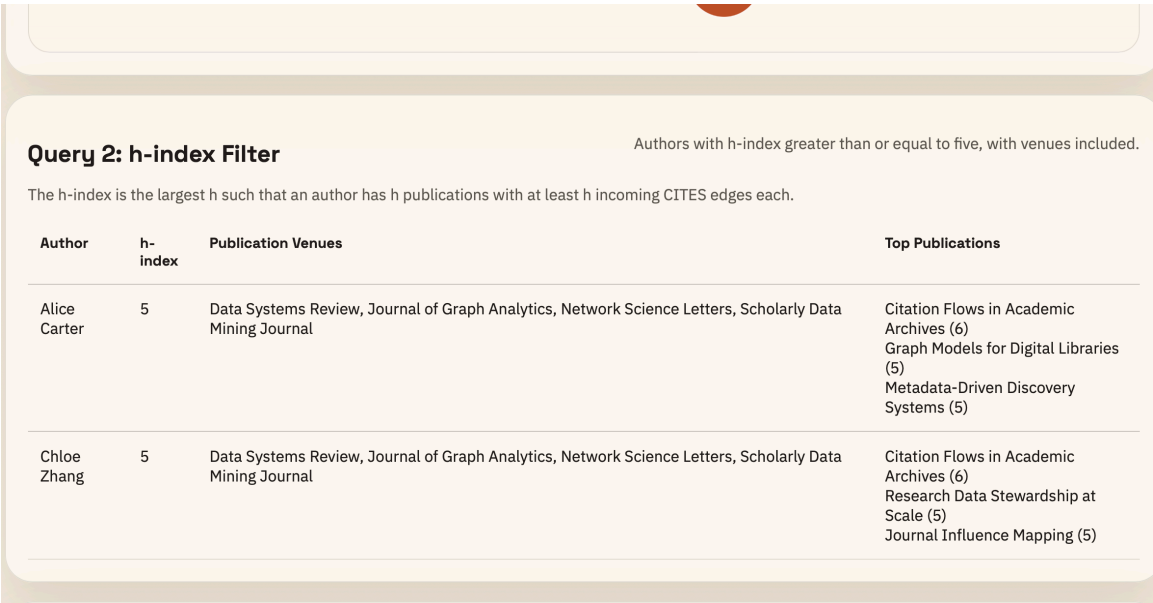

Figure 3. Author impact interface showing citation-based results.

7.3 Q1 Journal Influence Interface

The system also includes a Q1 journal influence section showing authors connected to authors who publish in Q1 venues, along with the related graph network.

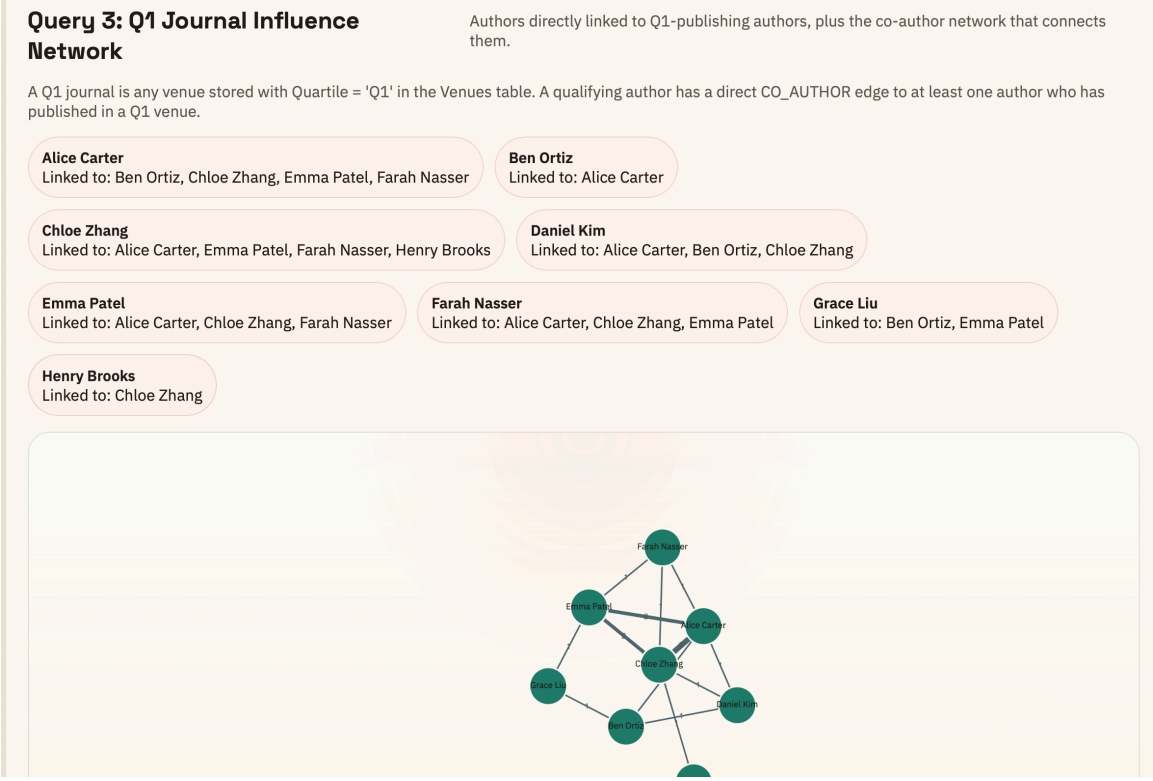

Figure 4. Q1 journal influence interface and corresponding graph network.

8. Implemented Functions

The current prototype already includes the main functions needed at this stage.

8.1 Database Initialization

The system can initialize the database schema and load sample academic data for testing and demonstration.

8.2 Co-Author Network Retrieval

The backend retrieves co-author relationships from the Edges table and returns the graph as nodes and edges for visualization.

8.3 Citation-Based Impact Analysis

The system computes author impact by counting incoming citation edges to authored publications and determining which authors meet the required threshold.

8.4 Q1 Journal Influence Analysis

The system identifies authors linked to Q1-publishing authors using venue quartile metadata and co-author edges.

These implemented functions demonstrate that the project is no longer only conceptual and now has a working analytical core.

9. Conclusion

Phase II has transformed the project from a conceptual graph design into a functional academic library prototype. The system now includes a detailed database schema, normalized subtype tables, multi-graph support, sample academic data, backend query processing, and browser-based graph visualization. The refinements made during implementation improved the flexibility and clarity of the design while keeping the system lightweight and practical. This Phase II prototype provides a strong foundation for later refinement, query expansion, and final demonstration.